Algorithmes distribués
ENSISA IR
Examen

En utilisant le même simulateur que lors du premier exercice, vous allez implémenter en Java l'algorithme **de l'écho de Chang et Segall, un algorithme de la vague**.

Cet algorithme sera utilisé dans le cadre d'un scénario particulier. Soit un système distribué fondé sur un réseau de capteurs de température. L'algorithme devra répondre à la question suivante : quelle sont les températures minimales et maximales ?

Première partie *(10 points)*

Un nœud (la station de base) lance la vague. Chaque capteur renvoie à son père le triplet (température minimale, température maximale, nombre de capteurs) de son sous-arbre.

Les processus choisissent une température au hasard au démarrage et l'affichent sur la console afin de pourvoir valider le calcul de la température minimale et maximale en fin d'exécution. L'initiateur calcule et affiche les valeurs minimale et maximale des températures lors de l'événement de décision.

Remarque

Un nœud reçoit deux sortes de messages : des **échos de ses fils**, qui portent des données, et des **jetons de non-fils**, qui arrivent par des arêtes transversales et ne portent rien d'utile. Les deux comptent pour atteindre les nœuds voisins, mais seuls les premiers doivent être agrégés. En résumé, seuls les messages émis en écho vers les pères portent les données.

Seconde partie *(10 points)*

Lorsque l'initiateur a calculé les valeurs minimale et maximale des températures, il relance l'algorithme de la vague pour diffuser ces valeurs à l'ensemble des capteurs. Chaque capteur affiche à la réception sa propre valeur de température et les valeurs minimale et maximale reçues.

Travaux

Ecrivez l'algorithme de l'écho de Chang et Segall en Java en intégrant le scénario présenté et en ciblant le simulateur utilisé lors de la première séance de TD.

Contexte

Vous utilisez le même simulateur que lors du premier exercice. Il figure dans le dossier "**TP 1 - Fichiers supports**" sur Moodle.

Rappels

Un processus est implémenté par une ***sous-classe*** de la classe ***Process*** du simulateur. Voir l'**API sur Moodle**.

Pour coder un processus, il faut typiquement redéfinir la méthode ***run()*** de la classe ***Process***. Cette méthode est appelée par le simulateur lorsque la méthode statique ***main()*** appelle la méthode ***start()*** de ***Process***. Ainsi si ***MyProcess*** est le nom de la classe qui implémente un processus, le source sera typiquement :

```
public class MyProcess extends Process {
    public MyProcess() throws IOException {
    }

    public static void main(String[] args) throws Exception {
        var process = new MyProcess();
        process.start();
    }

    public void run() throws Exception {
        // Code source du processus, que ce soit un initiateur ou un suiveur
    }

}
```

Pour compiler un source Java :

javac -cp processlibrary.jar ensisa/MyProcess.java

Pour lancer la simulation (4 est le nombre de processus (jusqu'à 20))

java -jar simulator.jar -n 4 -random -classpath ".:processlibrary.jar" ensisa.MyProcess

(Remplacez : par ; sous Windows)

Livrable

Vous déposerez vos sources Java sur Moodle à la fin de la séance.